

Aufgabe 1 · Strukturbasierte DBS

/6

Nennen Sie zwei Typen strukturbasierter Datenbanksysteme, Eigenschaften, Vor- & Nachteile.

Lösung: 2 von 4 Typen genügen:

- Hierarchische DB:** Baumstruktur, feste Eltern-Kind-Hierarchie; gut für hierarchische Infos (z.B. Organigramm). + sehr einfache Struktur. - keine Flexibilität. (heute kaum noch, außer Dateisysteme)
- Netzwerk-DB:** Erweiterung der hierarchischen DB um *mehrere* Elternelemente → keine strenge Hierarchie mehr. + hohe Flexibilität bei Abhängigkeiten. - wird schnell unübersichtlich.
- Relationale DB:** freie Strukturdefinition, Objekte als Relationen, Beziehungen über Schlüssel. + Struktur an Realität anpassbar. - jede Datenbasis eigene Struktur → Einarbeitung nötig.
- Objektorientierte DB:** analog zur OOP (Klassen, Vererbung, Objekte); komplexe Objekte gemeinsam gespeichert. + konzeptuell nah an OOP. - meist geringere Performance & Verbreitung.

Aufgabe 2 · GROUP_CONCAT & Join

/6 +/3

Tab1	
VorlesungsID	Vorlesungsname
1	Physik
3	Mathematik
2	Englisch
4	Informatik

Tab2	
VorlesungsID	Raumnummer
2	51.2.1
3	54.0.1
1	92.1.13
4	51.2.1
NULL	50.0.2

(a) Welches Ergebnis liefert das Statement?

SELECT GROUP_CONCAT(Vorlesungsname) AS Vorlesungsnamen, Raumnummer FROM Tab1 INNER JOIN Tab2 USING(VorlesungsID) GROUP BY Raumnummer ORDER BY Raumnummer ASC;

Lösung: INNER JOIN → NULL-Zeile (50.0.2) fällt weg. Nach Raumnummer sortiert:

Vorlesungsnamen	Raumnummer
Informatik,Englisch	51.2.1
Mathematik	54.0.1
Physik	92.1.13

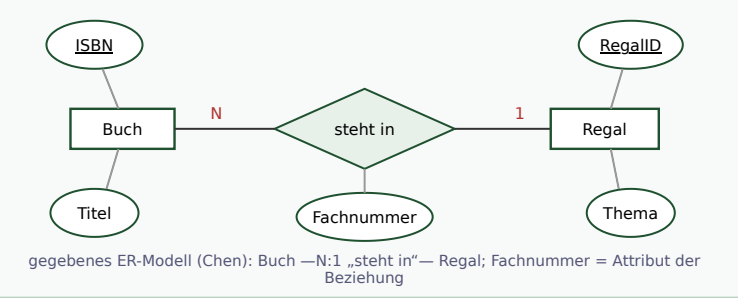
(b) So ändern, dass auch Räume ohne Vorlesung erscheinen. Begründung?

Lösung: INNER JOIN → RIGHT OUTER JOIN. FROM Tab1 RIGHT OUTER JOIN Tab2 USING(VorlesungsID) . Der RIGHT OUTER JOIN behält alle Datensätze der rechten Tabelle (Tab2), auch die von der linken nicht referenzierten → zusätzlich Zeile NULL | 50.0.2.

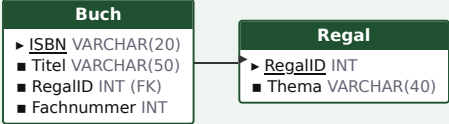
Aufgabe 3 · ER → relationales Modell

/5

Bibliothek: Zuordnung von Büchern zu Regalen. ER-Modell → relationales Modell (Datentypen/Optionalität egal).



Lösung: N:1-Beziehung → FK auf die N-Seite (Buch); das Beziehungsattribut Fachnummer wandert ebenfalls zur N-Seite.



Aufgabe 4 · Normalformen

/4 +/6

Gegebenes Modell:



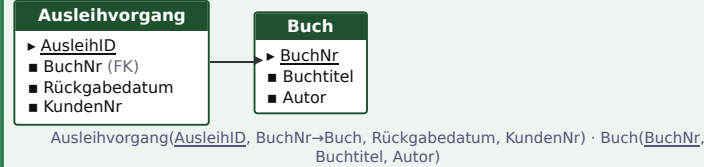
(a) In welcher Normalform? Begründung.

Lösung: 2. Normalform.

- 1.NF erfüllt (alle Attribute atomar).
- 2.NF automatisch erfüllt, da der einzige Schlüsselkandidat AusleihID einwertig ist → eine partielle Abhängigkeit ist strukturell unmöglich.
- Nicht 3.NF: Buchtitel, Autor hängen transitiv über BuchNr ab (AusleihID → BuchNr → Buchtitel, Autor).

(b) In die 3. NF überführen.

Lösung: Transitiv abhängige Attribute (Buchtitel, Autor) mit ihrem Determinanten BuchNr auslagern:



Zusatzaufgaben (eigene Übungen mit Lösung)

Weitere klausurtypische Aufgaben zu Themen der Vorlesung/Übung — Kontext: Tab1/Tab2 bzw. Bibliothek (Buch, Regal, Ausleihvorgang).

Z1 · LEFT vs. RIGHT OUTER JOIN. Geben Sie alle Vorlesungen aus – auch solche ohne Raum.

Lösung: SELECT Vorlesungsname, Raumnummer FROM Tab1 LEFT OUTER JOIN Tab2 USING(VorlesungsID); — LEFT behält alle Zeilen der linken Tabelle (Tab1). Spiegelbild zu 2(b): dort RIGHT (rechte Tabelle Tab2).

Z2 · Subquery + NULL-Fälle. Vorlesungen, die keinem Raum zugeordnet sind.

Lösung: SELECT Vorlesungsname FROM Tab1 WHERE VorlesungsID NOT IN (SELECT VorlesungsID FROM Tab2 WHERE VorlesungsID IS NOT NULL); — Falle: ohne IS NOT NULL liefert NOT IN bei einem NULL in der Subquery gar keine Zeilen (Vergleich mit NULL = UNKNOWN). Alternative: NOT EXISTS.

Z3 · GROUP BY ... HAVING. Räume, in denen mehr als eine Vorlesung stattfindet.

Lösung: SELECT Raumnummer, COUNT(*) AS Anzahl FROM Tab2 WHERE VorlesungsID IS NOT NULL GROUP BY Raumnummer HAVING COUNT(*) > 1; → 51.2.1 | 2. WHERE filtert Zeilen vor, HAVING Gruppen nach der Aggregation.

Z4 · Fremdschlüssel & referenzielle Optionen. Tab2 so anlegen, dass beim Löschen/Ändern einer Vorlesung die Raumzuordnungen mitgelöscht/aktualisiert werden.

CREATE TABLE Tab2 (VorlesungsID INT, Raumnummer VARCHAR(10), FOREIGN KEY(VorlesungsID) REFERENCES Tab1(VorlesungsID) ON UPDATE CASCADE ON DELETE CASCADE);

Lösung: RESTRICT würde Löschen/Ändern bei bestehenden Referenzen verbieten; CASCADE reicht die Aktion durch; SET NULL setzt den FK auf NULL.

Z5 · CHECK-Constraint. Fachnummer eines Buchs muss positiv sein.

Lösung: ALTER TABLE Buch ADD CONSTRAINT chk_fach CHECK (Fachnummer > 0); — CHECK prüft nur Werte derselben Zeile; zeilenübergreifende Regeln erfordern einen TRIGGER.

Z6 · Mengenoperationen. Unterschied UNION / INTERSECT / EXCEPT?

Lösung: UNION = Vereinigung (Duplikate entfernt, UNION ALL behält sie), INTERSECT = Schnittmenge (in beiden), EXCEPT = Differenz (im ersten, nicht im zweiten). Spalten müssen anzahl- und typkompatibel sein; ein finales ORDER BY/LIMIT gilt fürs Gesamtergebnis.

Z7 · View + CASE/Datum. View „offene Ausleihen“ mit Status (überfällig, falls Rückgabedatum in der Vergangenheit).

CREATE OR REPLACE VIEW offene_ausleihen AS SELECT AusleihID, KundenNr, Rückgabedatum, CASE WHEN Rückgabedatum < CURDATE() THEN 'überfällig' ELSE 'offen' END AS Status FROM Ausleihvorgang WHERE Rückgabedatum IS NULL OR Rückgabedatum >= '2000-01-01';

Lösung: Eine View ist eine gespeicherte Abfrage (kein eigener Speicher); CASE realisiert bedingte Ausgaben, CURDATE()/NOW() liefern das aktuelle Datum.

Z8 · Aggregat in Subquery. Vorlesung(en) mit der kleinsten VorlesungsID.

Lösung: SELECT Vorlesungsname FROM Tab1 WHERE VorlesungsID = (SELECT MIN(VorlesungsID) FROM Tab1); → Physik. Aggregatfunktionen (MIN/MAX/AVG) dürfen nicht direkt in WHERE stehen → als Subquery.

Z9 · Single-Row-Funktionen. Nur den Gebäudeteil der Raumnummer (vor dem ersten Punkt) ausgeben, in Großbuchstaben.

Lösung: SELECT UPPER(SUBSTRING_INDEX(Raumnummer, '.', 1)) AS Gebaeude FROM Tab2; — SUBSTRING_INDEX(s, trenn, n) schneidet am n-ten Trennzeichen; weitere: CONCAT, SUBSTR, REPLACE, LENGTH, ROUND.

Z10 · UPDATE mit Bedingung. Alle Räume im Gebäude 51 ins Gebäude 55 verlegen.

Lösung: UPDATE Tab2 SET Raumnummer = REPLACE(Raumnummer, '51.', '55.') WHERE Raumnummer LIKE '51.%'; — ohne WHERE würden alle Zeilen geändert (klassischer Fehler).

Z11 · INSERT-Varianten. Mehrere Zeilen einfügen; bei vorhandenem Schlüssel stattdessen aktualisieren.

Lösung: INSERT INTO Tab1 VALUES (5, 'Chemie'), (6, 'Biologie'); · Upsert: INSERT INTO Tab1 VALUES (5, 'Chemie') ON DUPLICATE KEY UPDATE Vorlesungsname='Chemie';

Z12 · ER-Erweiterung (Verständnis). Wie ändert sich das Bibliotheks-Modell, wenn ein Buch in mehreren Regalen (Exemplaren) stehen kann?

Lösung: Aus der N:1-Beziehung „steht in“ wird eine n:m-Beziehung → eigene Verbindungsrelation steht_in (ISBN, RegalID, Fachnummer); das Attribut Fachnummer gehört dann zur Verbindungsrelation (nicht mehr zu Buch).